Get Friendly with FFmpeg

Manipulating sound and video files appeals to one's creative instincts. FFmpeg, which is used by application software such as VLC, Mplayer, etc, can be used to great effect when working with sound and video files. Let's take a look at how concatenation, fading and overlay can be done in video files using FFmpeg.

'FFmpeg is a very fast video and audio converter that can also grab content from a live audio or video source. It can convert arbitrary sample rates and resize videos on-the-fly with a high quality polyphase filter.' This description and complete documentation on FFmpeg can be found at its home page. According to Wikipedia, FFmpeg is used by application software such as the VLC media player, MPlayer, Xine, HandBrake, Plex, Blender, YouTube and MPC-HC. It is one of the most popular open source audio/video converter software.

Understandably, FFmpeg is quite complex and it is naïve for the reader to expect or for the author to try to cover it in a single article. However, I would like to use a specific example to show how FFmpeg works.

In the January 2015 edition of *OSFY*, I had written about the tool 'grokit'. More details on the tool can be found at *http://grokit.pythonanywhere.com*. I had created a couple of videos explaining how this tool is used. These videos are accessible at *https://www.youtube.com/watch?v=XzrPlAfiC1w* and *https://www.youtube.com/watch?v=BTGnZShDiqA*. It is clear that these videos were created by stitching multiple screencast videos together. In this article, I will explain how this was accomplished using FFmpeg.

FFmpeg is available for Linux, Windows and Mac. It can be downloaded from *http://ffmpeg.org/download.html*.

Stitching two videos

Let us assume that we have two AVI (1920 x 1080) files with names *video1.avi* and *video2.avi*. Let the files be of duration 5s and 10s, respectively. The following command will create *video.avi* by concatenating the two videos and fading the videos at the point where they are concatenated. Note that there are multiple ways to achieve the same effect and the command used below is merely one example. Please refer to *https://www.ffmpeg.org/ffmpeg-all.html* for a detailed explanation of all the options used in the command.

```
ffmpeg -i video1.avi -i video2.avi -f lavfi -i color=black
-filter_complex "[0:v]fade=t=out:st=4:d=1:alpha=1,setpts=P
TS-STARTPTS[v0];[1:v]fade=t=in:st=0:d=1:alpha=1,setpts=PTS-
STARTPTS+5/TB[v1];[2:v]scale=1920x1080,trim=duration=15[base];
[base][v0]overlay[base1];[base1][v1]overlay[outv]" -map [outv]
video.avi
```

Admittedly, the above command looks scary.

I will try to explain the command, starting with the options used in it:

- *–i:* FFmpeg uses the '*-i*' option to accept input files.